

TEMA N° 1



TEMA 1 PROGRAMACIÓN ORIENTADA A OBJETOS (POO AVANZADA)



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Dominar** los pilares avanzados de la Programación Orientada a Objetos (POO) para estructurar sistemas de software escalables y reutilizables.
2. **Aplicar** conceptos como herencia múltiple, polimorfismo y clases abstractas en la resolución de problemas tecnológicos reales de nuestra comunidad en Pailón.
3. **Desarrollar** una mentalidad analítica propia de un Técnico en Sistemas Informáticos, conectando el código con arquitecturas de software modernas, bases de datos y la inteligencia artificial.



PRÁCTICA



¿Qué pasaría si tuviéramos que programar desde cero el comportamiento de cada negocio, casa y parque de nuestro municipio sin poder reutilizar ni una sola línea de código?

Imagina la siguiente situación en la población de Pailón: El CEA Herman Ortiz Camargo, desde su nuevo y flamante módulo en el Barrio Saó, ha sido convocado por el municipio para desarrollar un "Sistema Integral de Gestión Urbana". Este sistema debe registrar y administrar las actividades de diferentes sectores: el intenso comercio formal cerca de la iglesia en el **Barrio Central Fátima**, el flujo de transporte y actividades en la estación de tren y campos deportivos del **Barrio 24 de Septiembre**, y el mantenimiento de los parques infantiles del **Barrio Residencial Norte**.

Si intentamos programar esto usando programación estructurada básica, tendríamos que escribir funciones separadas y repetitivas para cobrar impuestos a un mercado, para registrar el mantenimiento de un parque o para agendar partidos en una cancha. El código se volvería gigantesco, difícil de leer e imposible de mantener.

¿Cómo podemos estructurar nuestro código para que un "Campo Deportivo" y un "Local Comercial" compartan características comunes (como tener una dirección y un encargado) pero mantengan comportamientos completamente distintos (como organizar torneos vs. vender productos), escribiendo el código de esas características compartidas una sola vez?



TEORÍA



En semestres anteriores, aprendiste que la POO nos permite modelar el mundo real creando "Clases" (plantillas) y "Objetos" (instancias de esas plantillas). Ahora, como Técnico en Sistemas Informáticos, daremos el paso hacia arquitecturas profesionales. Usaremos Python en nuestros ejemplos por su claridad y soporte nativo para características avanzadas.



2.1. CONCEPTO E IMPLEMENTACIÓN DE LA HERENCIA DE CLASES

La herencia es el mecanismo por el cual una clase nueva (clase hija o subclase) se crea a partir de una clase existente (clase padre o superclase). La clase hija hereda todos los atributos (variables) y métodos (funciones) de la clase padre, permitiendo la reutilización masiva de código.

Imagina que todas las edificaciones en Pailón comparten datos básicos. Creamos una clase padre y luego clases hijas específicas.



Python

```

# Definimos la clase padre o superclase
class InfraestructuraUrbana:
    # El método __init__ inicializa los atributos básicos de la infraestructura
    def __init__(self, nombre, barrio, encargado):
        # Asignamos el nombre de la infraestructura
        self.nombre = nombre
        # Asignamos el barrio donde se ubica (ej. Barrio 24 de Septiembre)
        self.barrio = barrio
        # Asignamos la persona responsable
        self.encargado = encargado
    # Método general que cualquier infraestructura puede usar
    def mostrar_info(self):
        # Imprimimos en consola los datos generales
        print(f"{{
self.barrio}}
}}
self.nombre}
- Responsable: {{
self.encargado}}
")
# Definimos la clase hija que HEREDA de InfraestructuraUrbana
class
  
```



```

CampoDeportivo(InfraestructuraUrbana):    # Inicializamos la clase hija    def
__init__(self, nombre, barrio, encargado, tipo_cancha):    # Llamamos al
constructor de la clase padre para no repetir código
InfraestructuraUrbana.__init__(self, nombre, barrio, encargado)    #
Añadimos un atributo exclusivo del campo deportivo    self.tipo_cancha =
tipo_cancha    # Método exclusivo de la clase hija    def
organizar_partido(self, equipo1, equipo2):    # Imprimimos la acción
específica usando el atributo heredado    print(f"
Partido programado en {
self.nombre}
: {
equipo1}
vs {
equipo2}
")
# Creamos un objeto de la clase hijacancha_24_sept = CampoDeportivo("
Cancha Principal", "
Barrio 24 de Septiembre", "
Juan Pérez", "
Fútbol Sala")
# Usamos el método heredado de la clase padrecancha_24_sept.mostrar_info()
# Usamos el método propio de la clase hijacancha_24_sept.organizar_partido("
Dep. Fátima", "
Saó FC")

```

2.2. PALABRA RESERVADA SUPER Y HERENCIA MÚLTIPLE

La función `super()` nos da acceso temporal a los métodos de la clase padre, lo que es mucho más limpio y seguro que llamar a la clase padre por su nombre directamente (como hicimos en el ejemplo anterior).

Además, algunos lenguajes como Python o C++ permiten la **Herencia Múltiple**, donde una clase hija puede heredar de *dos o más* clases padres al mismo tiempo. Esto es útil para combinar comportamientos. Pensemos en un negocio en el **Barrio Central Fátima** que también funciona como hogar para una familia.

Python

```

# Definimos la primera clase padreclass LocalComercial:    # Constructor del
Local comercial    def __init__(self, nit, rubro):    # Guardamos el NIT
self.nit = nit    # Guardamos el tipo de negocio    self.rubro = rubro
# Método para realizar una venta    def vender(self):    # Mensaje simulando
una venta    print(f"
Venta registrada en el rubro de {

```



```

self.rubro}
.")
# Definimos la segunda clase padreclass Vivienda:      # Constructor de la
vivienda      def __init__(self, numero_habitantes):      # Guardamos cuántas
personas viven ahí      self.numero_habitantes = numero_habitantes      #
Método para descansar      def descansar(self):      # Mensaje simulando la
acción de descanso      print("
La familia está descansando.")
# Clase hija que hereda de AMBAS clases (Herencia Múltiple)
class NegocioFamiliar(LocalComercial, Vivienda):      # Constructor que recibe
todos los parámetros necesarios      def __init__(self, nit, rubro,
numero_habitantes, familia):      # Usamos super() para inicializar la primera
clase padre (LocalComercial)      super().__init__(nit, rubro)      #
Inicializamos explícitamente la segunda clase padre (Vivienda)
Vivienda.__init__(self, numero_habitantes)      # Atributo propio de la clase
hija      self.familia = familia
# Instanciamos el negocio familiartienda_fatima = NegocioFamiliar("1234567015",
"
Abarrotes", 5, "
Familia Condori")
# Usamos un método de La primera clase padretienda_fatima.vender()
# Usamos un método de La segunda clase padretienda_fatima.descansar()

```

2.3. COMPRENSIÓN Y APLICACIÓN DEL POLIMORFISMO

El término "Polimorfismo" significa "muchas formas". En programación, permite que objetos de diferentes clases sean tratados como si fueran objetos de la misma clase. Diferentes clases pueden tener un método con el mismo nombre, pero cada una lo ejecuta a su manera.

Imagina que el municipio necesita calcular el mantenimiento de distintas áreas. Un coliseo requiere un cálculo distinto al de un parque, pero para el sistema, la instrucción debe ser simplemente "calcular mantenimiento".



Python

```
# Clase para el Coliseo
class ColiseoMunicipal:
    def calcular_mantenimiento(self):
        # Retorna el costo basado en limpieza y energía
        return "Costo Coliseo: Limpieza de gradas + Iluminación de cancha."

# Clase para el Parque Infantil
class ParqueInfantil:
    def calcular_mantenimiento(self):
        # Mismo nombre de método, diferente comportamiento
        # Retorna el costo basado en pintura y jardinería
        return "Costo Parque: Pintura de columpios + Corte de césped."

# Función que aplica polimorfismo, recibe CUALQUIER infraestructura
def generar_reporte_mantenimiento(infraestructura):
    # Llama al método sin importar de qué clase sea el objeto
    print(infraestructura.calcular_mantenimiento())

# Instanciamos objetos de los diferentes barrios
coliseo_27_mayo = ColiseoMunicipal()
parque_norte = ParqueInfantil()

# Pasamos objetos distintos a la misma función (Polimorfismo en acción)
generar_reporte_mantenimiento(coliseo_27_mayo)
generar_reporte_mantenimiento(parque_norte)
```

2.4. SOBRECARGA DE MÉTODOS Y SOBREESCRITURA (OVERRIDE)

Sobreescritura (Override): Ocurre cuando una clase hija redefine un método que heredó de su clase padre para cambiar su comportamiento.



Sobrecarga (Overload): Consiste en tener múltiples métodos con el mismo nombre pero con diferentes parámetros (distinta cantidad o tipo). *Nota: Python no soporta la sobrecarga clásica como Java o C#, pero la simulamos usando parámetros por defecto.*

Python

```
# -
-- EJEMPLO DE SOBREENSCRITURA (OVERRIDE) -
--# Clase padreclass UsuarioCEA:    # Método original    def
acceder_sistema(self):    # Acceso básico    print("
Acceso concedido al portal público del CEA.")
# Clase hijaclass EstudianteSistemas(UsuarioCEA):    # Sobreescribimos el método
heredado    def acceder_sistema(self):    # Comportamiento modificado y
especializado    print("
Acceso concedido a los laboratorios virtuales de programación del CEA.")
# -
-- EJEMPLO DE SOBRECARGA SIMULADA (OVERLOAD) -
--class GestorMatricula:    # Método con parámetros opcionales para simular
sobrecarga    def registrar_estudiante(self, nombre, modulo, beca=None):
# Si no se envía el parámetro beca    if beca is None:    # Mensaje
estándar    print(f"
Estudiante {
nombre}
registrado en Módulo {
modulo}
. Pago regular.")    # Si se envía el parámetro beca    else:
# Mensaje modificado    print(f"
Estudiante {
nombre}
registrado en Módulo {
modulo}
. Beca aplicada: {
beca}
%.")
# Instanciamos el gestor gestor = GestorMatricula()
# Llamada 1: como si fuera un método de 2
parámetros gestor.registrar_estudiante("
María", "
XIII")
# Llamada 2: como si fuera un método de 3
parámetros gestor.registrar_estudiante("
Carlos", "
XIII", 50)
```



2.5. DECLARACIÓN Y USO DE CLASES ABSTRACTAS E INTERFACES

Una **Clase Abstracta** es un modelo que no se puede instanciar directamente (no puedes crear objetos de ella), sino que sirve obligatoriamente como base para otras clases. Define métodos que las clases hijas *deben* construir sí o sí (contrato). En Python se usa la librería abc (Abstract Base Classes). Las **Interfaces** son similares, pero en lenguajes como Java son contratos puros donde todos los métodos son abstractos.

Si desarrollamos un sistema de pagos para los comercios de **San Expedito** o **Las Misiones**, no podemos permitir un "PagoGenérico". Debe ser obligatoriamente Efectivo, QR o Tarjeta.

Python

```
# Importamos las herramientas para crear clases abstractas en Python
from abc import ABC, abstractmethod
# Definimos la clase abstracta (Actúa como una Interfaz estricta)
class MetodoPago(ABC): # El decorador obliga a las clases hijas a implementar este método
    @abstractmethod def procesar_pago(self, monto): # No se pone código funcional aquí, solo la palabra 'pass'
    pass
# Clase hija 1 que cumple el contrato
class PagoQR(MetodoPago): # Implementación OBLIGATORIA del método abstracto
    def procesar_pago(self, monto): # Lógica para procesar el pago por QR
        print(f"Generando código QR para cobrar Bs. {monto}")
# Clase hija 2 que cumple el contrato
class PagoEfectivo(MetodoPago): # Implementación OBLIGATORIA del método abstracto
    def procesar_pago(self, monto): # Lógica para procesar el pago en efectivo
        print(f"Abriendo caja registradora para recibir Bs. {monto}")
# Si intentamos: pago = MetodoPago() -> DARÁ ERROR, no se puede instanciar
# Instanciamos las clases hijas correctamente
compra_barrio = PagoQR()
# Ejecutamos el método
compra_barrio.procesar_pago(150)
```

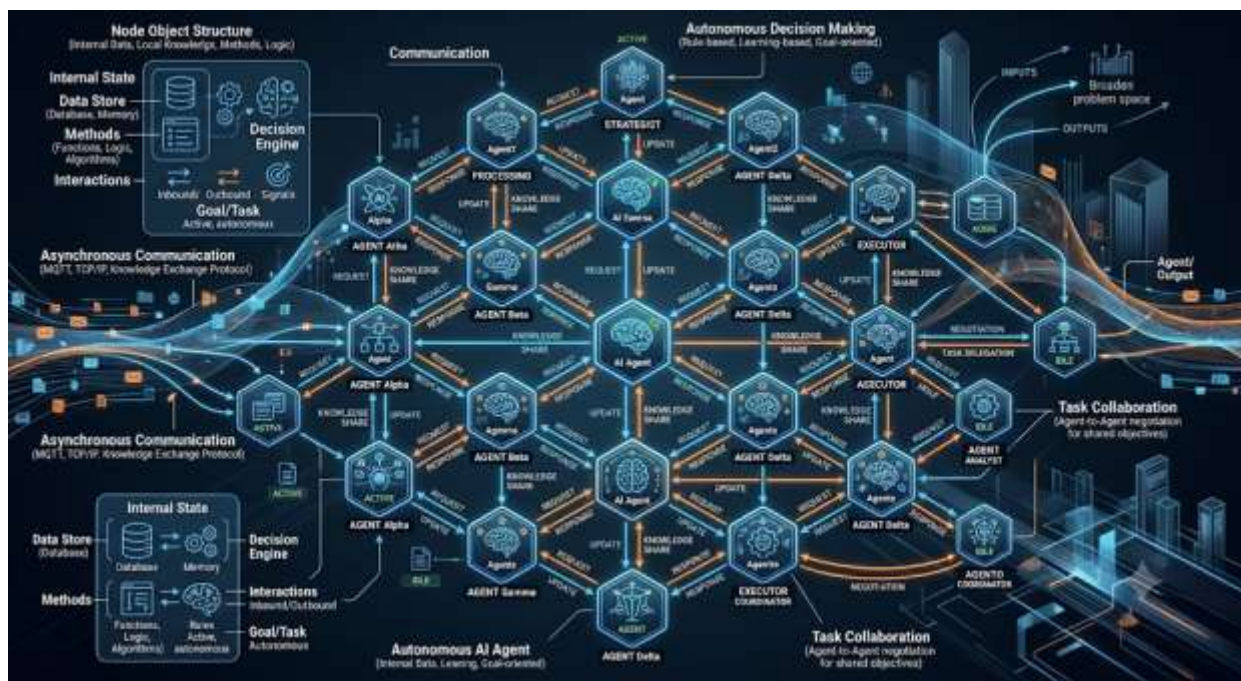
2.6. INTEGRACIÓN DE POO CON BASES DE DATOS EN LA NUBE (TENDENCIA AVANZADA)

Como Técnico en Sistemas Informáticos, no solo harás scripts locales. La POO moderna se enlaza directamente con bases de datos en la nube (como PostgreSQL en plataformas como Render o



Railway) a través de ORMs (Object-Relational Mapping). Un ORM convierte una tabla de base de datos en una Clase de POO. Así, guardar un dato es tan sencillo como hacer objeto.save(), evitando escribir complejas consultas SQL a mano. Esto asegura que la lógica de negocio y los datos vivan en armonía bajo el paradigma orientado a objetos.

2.7. POO EN LA ERA DE LA INTELIGENCIA ARTIFICIAL: AGENTES Y OBJETOS (TENDENCIA AVANZADA)



La Inteligencia Artificial está redefiniendo la POO. Hoy en día, desarrollamos "Agentes" de IA. En código, un Agente se programa exactamente como un Objeto en POO: tiene "Estado" (memoria de interacciones pasadas, contexto) y "Comportamientos" (métodos para pensar, buscar en la web, o generar respuestas). Comprender a fondo las clases y el polimorfismo es el primer paso obligatorio para que mañana puedas programar agentes autónomos que asistan en la educación o la administración de los módulos de un CEA.



2.8. ARQUITECTURA DE MICROSERVICIOS ORIENTADA A OBJETOS (TENDENCIA AVANZADA)

El software actual ya no se construye como un bloque gigante (Monolito). Se divide en pequeños servicios independientes llamados Microservicios. La POO es la base teórica de esto: cada microservicio actúa como un "objeto" a gran escala en la nube. Tiene sus propios datos privados (encapsulamiento) y se comunica con otros microservicios mediante interfaces bien definidas (APIs), aplicando los principios de bajo acoplamiento y alta cohesión que fundamentan el diseño de clases que acabamos de estudiar.

CIERRE TEÓRICO OBLIGATORIO

Mitos sobre la POO	Realidades para el Técnico en Sistemas
Mito 1: "La herencia profunda (muchos niveles de padres e hijos) siempre es la mejor forma de reutilizar código."	Realidad: Abusar de la herencia crea sistemas rígidos y frágiles. A menudo, la <i>composición</i> (un objeto que contiene a otros objetos) es más flexible y recomendada en el desarrollo moderno.
Mito 2: "Las clases abstractas e interfaces son solo teoría inútil que complica el código innecesariamente."	Realidad: Son la columna vertebral del trabajo en equipo. Establecen "contratos" obligatorios que evitan que otros programadores rompan el sistema al añadir nuevas funciones o métodos de pago.

□ Glosario de Términos:

1. **Clase Base (Superclase):** Plantilla principal de la cual otras clases derivan sus características y comportamientos fundamentales.
2. **Instanciación:** El proceso técnico de asignar memoria y crear un objeto concreto a partir de una clase abstracta o regular.
3. **ORM (Object-Relational Mapping):** Técnica de programación que convierte datos entre sistemas de bases de datos incompatibles y lenguajes orientados a objetos.
4. **Acoplamiento:** Medida de la dependencia entre los diferentes módulos o clases de un sistema de software. En POO buscamos un bajo acoplamiento.



5. **Firma de Método:** La combinación del nombre del método y el número/tipo de sus parámetros. Es fundamental para la sobrecarga.

□ **Ponte a Prueba:**

1. ¿Qué concepto permite que la clase Moto y la clase Camion ejecuten el método acelerar() de forma completamente distinta usando el mismo comando?

1. a) Herencia Múltiple
2. b) Polimorfismo
3. c) Encapsulamiento

2. En Python, si una clase tiene métodos vacíos decorados con @abstractmethod y exige que sus hijas los programen, estamos hablando de:

1. a) Una Clase Abstracta / Interfaz
2. b) Un ORM de base de datos
3. c) Un Objeto Instanciado

3. ¿Cuál es el propósito principal de la función super() en la programación orientada a objetos?

1. a) Crear funciones más rápidas en el procesador.
2. b) Eliminar la clase hija de la memoria RAM.
3. c) Llamar a los métodos y constructores de la clase padre sin nombrarla explícitamente.

(Respuestas: 1-b, 2-a, 3-c)



VALORACIÓN



Como Técnico en Sistemas Informáticos, tu responsabilidad va más allá de escribir código que funcione; debes escribir código que perdure y cuide los recursos. Al utilizar herramientas avanzadas de la POO como el polimorfismo y la herencia correcta, creamos sistemas **modulares**.



¿Por qué importa esto a nivel social y medioambiental? Un código espagueti y mal estructurado requiere mayor capacidad de procesamiento, saturando servidores y consumiendo más energía eléctrica injustificadamente. Además, cuando el software municipal de Pailón necesite actualizarse (por ejemplo, para agregar el **Barrio Ovidio Barberi** o **Jardines de Pailón** al mapa digital), un diseño basado en clases abstractas e interfaces permitirá añadir el nuevo módulo sin tener que desechar ni reescribir el sistema entero. Esto reduce la obsolescencia programada a nivel de software y evita que nuestras instituciones tengan que gastar recursos públicos constantemente en comprar nuevos sistemas extranjeros cerrados. Un código eficiente, limpio y bien arquitectado es, en esencia, un acto de responsabilidad profesional y ética tecnológica con tu municipio y el medio ambiente.



PRODUCCIÓN



Reto Final: "Sistema Cívico Pailón"

Tu misión es aplicar todo lo aprendido creando un script en Python que gestione entidades del municipio aplicando Herencia, Sobreescritura y Polimorfismo.

Paso a Paso:

1. **Abre tu entorno de desarrollo** (VS Code, Pycharm o un editor online).
2. **Crea la Superclase Abstracta:** Importa el módulo abc. Crea una clase EntidadMunicipal(ABC). Define en el constructor los atributos nombre y barrio. Crea un método abstracto llamado calcular_presupuesto_anual(self).
3. **Aplica Herencia y Sobreescritura:**
 1. Crea una clase UnidadEducativa que herede de EntidadMunicipal. Añade el atributo cantidad_estudiantes. Sobreescribe el método abstracto: el presupuesto será cantidad_estudiantes * 50.



2. Crea una clase CentroSalud que herede de EntidadMunicipal. Añade el atributo cantidad_camillas. Sobreescribe el método abstracto: el presupuesto será cantidad_camillas * 1000.
4. **Aplica Polimorfismo:** Crea una función independiente llamada mostrar_presupuestos(lista_entidades). Esta función debe recorrer la lista con un bucle for e imprimir el nombre de la entidad, su barrio y el resultado de llamar a su método calcular_presupuesto_anual().
5. **Prueba el Código:** Instancia al "CEA Herman Ortiz Camargo" (Barrio Saó, 150 estudiantes) usando la clase UnidadEducativa. Instancia la "Posta San Antonio" (Barrio San Antonio, 10 camillas) usando la clase CentroSalud. Pásalos a tu función en una lista.
6. **Comenta tu código:** Asegúrate de explicar qué pilar de la POO estás utilizando en cada bloque. ¡Ejecuta y verifica que no haya errores!

5. RECURSOS ADICIONALES

Para seguir puliendo tus habilidades como Técnico y profundizar en la arquitectura de software, te recomiendo explorar las siguientes fuentes oficiales y comunitarias:

1. **Documentación Oficial de Python (Sección Clases):** La guía definitiva y directa de los creadores del lenguaje para entender los entresijos de la herencia y métodos especiales. *(Busca en Google: "Python 3 Classes Official Documentation")*
2. **Refactoring Guru (Patrones de Diseño):** Excelente sitio visual que explica cómo los conceptos avanzados de POO (Interfaces, Clases Abstractas) se unen para resolver problemas comunes de arquitectura de software. *(Busca en Google: "Refactoring Guru Patrones de Diseño")*
3. **GitHub (Exploración de Código Abierto):** Te invito a buscar repositorios de proyectos reales contruidos con frameworks como Django (que usa POO intensivamente en sus modelos y vistas). Leer código profesional estructurado es la mejor forma de mejorar. *(Busca en GitHub: "Django open source projects")*